



Instituto Tecnológico de Mazatlán

Unidad de Medición Inercial (IMU 6-DOF MPU6050): Lectura directa de acelerómetros y giróscopos.

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Barrón Hernández

Integrantes:

Ximena Armenta Dávila

Carlos Tadeo Ibarra Castañeda

Dulce Princesa Ulivarría González

1. Introducción

La unidad de medición inercial MPU-6050 integra un acelerómetro de tres ejes y un giroscopio de tres ejes. Por eso se conoce como una IMU de seis grados de libertad (6-DOF). El acelerómetro registra aceleraciones, incluida la referencia de la gravedad, mientras que el giroscopio mide la velocidad de giro. Al combinar ambas lecturas es posible estimar la inclinación con mayor estabilidad que usando un solo sensor [1, 2].

En esta práctica se implementó un inclinómetro con Arduino UNO R4 WiFi y lectura directa de los registros del MPU-6050 mediante I2C. Se utilizó el ángulo frontal (*pitch*), correspondiente al giro alrededor del eje Y, para controlar un motorreductor con un puente H L298N: el signo del ángulo determina la dirección y su magnitud determina el PWM. El programa incorpora calibración, filtro complementario, zona muerta, rampa de aceleración y paro de seguridad.

2. Objetivos

Objetivo general:

- Medir la inclinación frontal con el MPU-6050 y usarla para controlar el sentido y la velocidad de un motorreductor de forma gradual y segura.

Objetivos específicos:

- Comunicar el sensor con el Arduino por I2C y comprobar su identificación WHO_AM_I.
- Leer el acelerómetro y el giroscopio directamente, configurar sus rangos y calibrar el eje Y.
- Combinar las lecturas mediante un filtro complementario y aplicar una zona muerta de $\pm 5^\circ$.
- Controlar el L298N con PWM, desacelerar antes de invertir y detener la orden del motor ante una falla.
- Coordinar cuatro tareas con `millis()` y mostrar la inclinación en el Monitor Serie y la matriz LED.

3. Herramientas y material utilizado

- 1 Arduino UNO R4 WiFi y cable USB-C de datos.
- 1 módulo MPU-6050 (GY-521) y cables de conexión Dupont.
- 1 módulo puente H L298N con disipador.
- 1 motorreductor de corriente directa con rueda.
- 1 pila de 9 V y conexiones para la alimentación externa del motor.
- Computadora con Arduino IDE y paquete Arduino UNO R4 Boards.
- Bibliotecas Wire y Arduino_LED_Matrix, incluidas con la placa; Monitor Serie a 115200 baudios.

La protoboard es opcional para organizar las señales y la tierra común. La corriente del motor se lleva directamente a los bornes del puente H.

4. Desarrollo / Metodología

Se preparó el Arduino IDE para la placa UNO R4 WiFi y se conectó el MPU-6050 al bus I2C principal. SDA se asignó a A4 y SCL a A5; también pueden usarse los pines SDA y SCL junto a AREF, porque pertenecen al mismo bus `Wire`. El conector Qwiic corresponde a otro bus, `Wire1` [3].

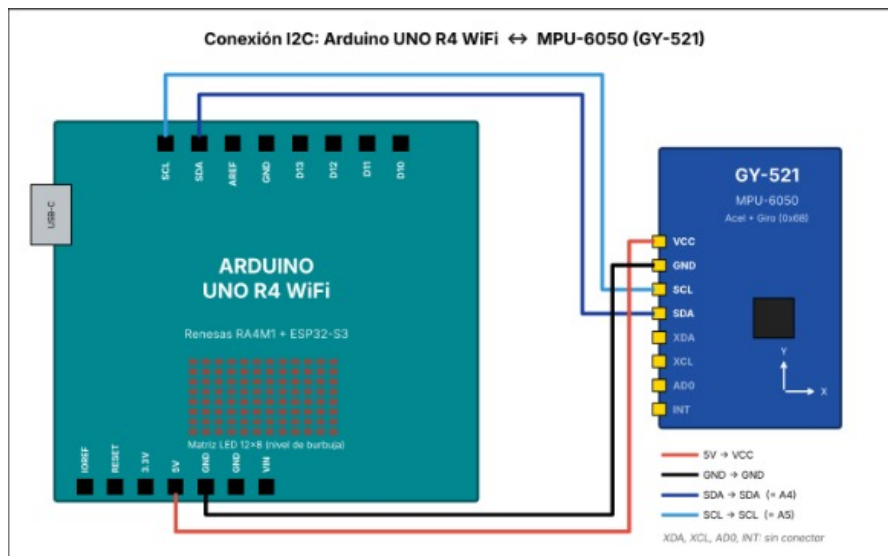


Figura 1. Diagrama proporcionado para la conexión I2C entre el UNO R4 WiFi y el MPU-6050.

Tabla I. Conexiones de señal del sensor y del puente H.

Elemento / pin	Arduino o conexión	Función
MPU-6050 SDA	A4 / SDA	Datos del bus I2C.
MPU-6050 SCL	A5 / SCL	Reloj del bus I2C.
MPU-6050 GND	GND común	Referencia eléctrica.
MPU-6050 AD0	Nivel bajo / GND	Seleccionar dirección I2C 0x68.
L298N ENA	D9 (PWM)	Regular la potencia del canal A.
L298N IN1 / IN2	D8 / D7	Seleccionar el sentido de giro.
L298N OUT1 / OUT2	Dos cables del motor	Salida de potencia al motorreductor.

El diagrama muestra 5 V en VCC del **módulo** GY-521. Esto requiere una placa con regulador que acepte esa entrada; el chip MPU-6050 trabaja alrededor de 3.3 V. También se debe comprobar la compatibilidad de SDA y SCL con la lógica del Arduino y adaptar los niveles si el módulo lo requiere. INT, XDA y XCL no se utilizan.

El Arduino se alimentó por USB y la pila de 9 V se destinó a la entrada de potencia del L298N, con tierra común entre Arduino, sensor, puente y fuente. Para controlar ENA se retira su jumper. En el esquema de alimentación lógica propuesto, el jumper 5V-EN se retira antes de alimentar la lógica del módulo desde 5 V del Arduino. La pila rectangular tiene capacidad limitada para un motor; no asegura el mismo rendimiento que una fuente dimensionada para su corriente.

Organización del programa

El programa `Inclinometro_Motor_R4.ino` deja el motor deshabilitado antes de iniciar el sensor. Lee `WHO_AM_I` en el registro `0x75` y exige el valor `0x68`. Después despierta el MPU-6050 y configura los rangos de ± 2 g para aceleración y ± 250 °/s para giro. La dirección I2C y `WHO_AM_I` son datos distintos: si `AD0` cambia la dirección a `0x69`, la identificación esperada sigue siendo `0x68` [2].

Con el sensor quieto se toman 500 lecturas del giroscopio Y durante unos cinco segundos. Su promedio es el *offset* que se resta a las lecturas siguientes. El ángulo inicial se calcula con el acelerómetro para que el filtro no arranque desde una inclinación ficticia de cero.

La lectura directa convierte los datos de aceleración con 16384 cuentas por g y los del giroscopio con 131 cuentas por °/s, según los rangos elegidos. El filtro complementario combina el cambio de ángulo integrado del giroscopio con la referencia de inclinación del acelerómetro. En el código su peso se ajusta al intervalo real entre lecturas, con una constante de tiempo de 0.5 s.

Tabla II. Tareas programadas en el ciclo principal.

Tarea	Intervalo	Acción
Sensor y filtro	10 ms	Obtener pitch y calcular el PWM objetivo.
Rampa y motor	20 ms	Acercar el PWM aplicado al objetivo.
Monitor Serie	500 ms	Informar solo si cambia el estado mostrado.
Matriz LED	50 ms	Actualizar el punto o el marco de centro.

La zona muerta fija un objetivo de cero cuando $|\text{pitch}|$ es menor que 5° . Desde 5° hasta 45° el PWM objetivo aumenta de 90 a 255; desde 45° queda limitado al máximo. El signo positivo indica adelante y el negativo, reversa. La rampa cambia 9 unidades de PWM cada 20 ms, por lo que tarda aproximadamente 580 ms en pasar de 0 a 255. Antes de invertir, el PWM pasa por cero.

La siguiente estructura permite atender cada tarea a su propio ritmo:

```
if (ahora - ultimoSensor >= 10) {  
    ultimoSensor = ahora; actualizarSensor();  
}  
if (ahora - ultimoMotor >= 20) {  
    ultimoMotor = ahora; actualizarRampa();  
}  
if (ahora - ultimoSerie >= 500) {  
    ultimoSerie = ahora; informarCambios();  
}  
if (ahora - ultimaMatriz >= 50) {  
    ultimaMatriz = ahora; actualizarMatriz();  
}
```

Si una lectura falla, ENA se pone a cero sin rampa y se anuncia **PARO DE SEGURIDAD**. El programa vuelve a buscar el sensor y recupera el control desde PWM cero cuando obtiene una lectura válida. La matriz presenta un punto de 2×2 LEDs según el ángulo, un marco dentro de $\pm 2^\circ$ y una X en caso de falla. Las esperas con `delay()` se limitan al arranque y la calibración, con el motor apagado.

5. Evidencia del circuito funcionando

La Figura 2 muestra el armado con el Arduino UNO R4 WiFi, el puente H L298N y el motorreductor con rueda. La placa está conectada a la computadora mediante USB y su matriz LED está encendida. Los cables del motor llegan al puente H, que recibe las señales de dirección y PWM del Arduino.

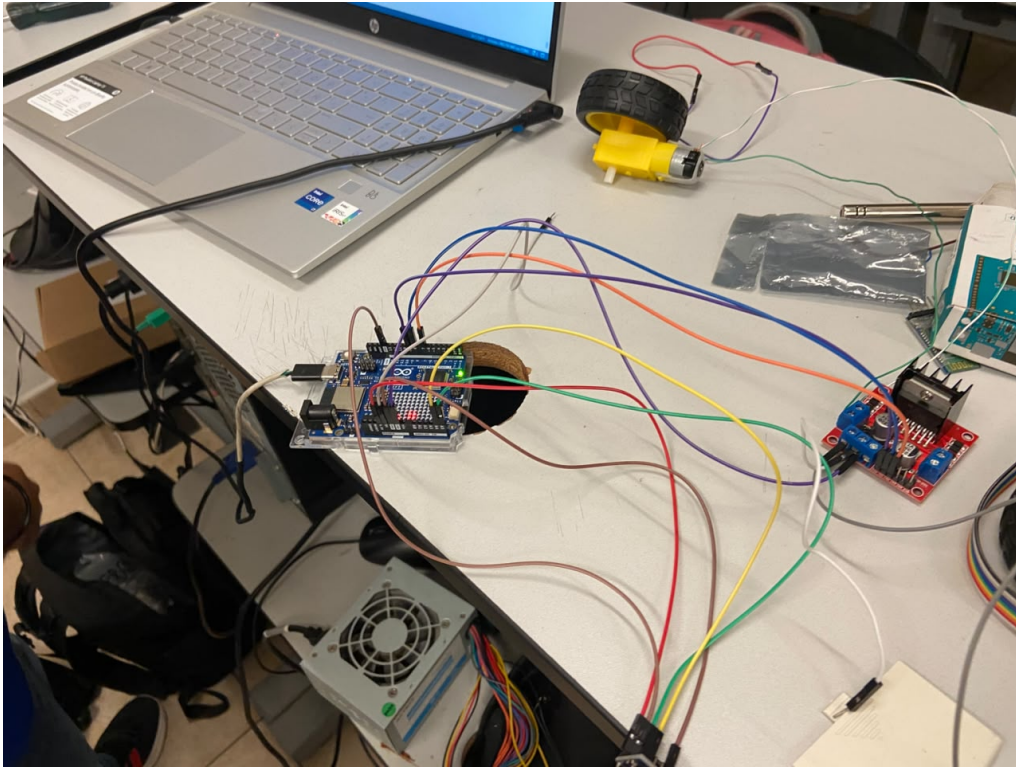


Figura 2. Montaje de la práctica con Arduino UNO R4 WiFi, L298N y motorreductor con rueda.

La inclinación se representa mediante el sensor y se convierte en una orden de movimiento. En el Monitor Serie se reportan los grados, la intensidad de inclinación, el sentido del motor y el porcentaje de PWM aplicado. Esta información permite relacionar el movimiento del sensor con la orden enviada al puente, sin añadir una pantalla externa.

Durante la puesta en marcha se presentó el aviso “ERROR INICIAL: MPU-6050 no responde 0x68 o no se configura”. Este mensaje mantiene el motor deshabilitado cuando no se completa la identificación o la configuración. Para localizar el problema se preparó una prueba I2C que busca dispositivos en Wire y Wire1 y lee WHO_AM_I en las direcciones 0x68 y 0x69. La captura de funcionamiento de la Figura 3 ya muestra lecturas de inclinación y estados del motor.

El centro de la matriz y la zona muerta del motor no tienen el mismo rango: el marco aparece dentro de $\pm 2^\circ$, mientras que el objetivo del motor es cero para magnitudes menores que 5° . Al llegar al centro, una orden de giro previa puede tardar brevemente en bajar a cero por la rampa normal.

Lecturas y estados en el Monitor Serie

La captura incluye ángulos positivos y negativos, porcentajes de PWM intermedios, saturación al 100 % y avisos de entrada y salida del centro. El último registro muestra 0°, motor detenido y PWM de 0 %.

Inclinacion:	adelante	5 grados	leve	Motor:	adelante	PWM: 14 %
Inclinacion:	adelante	5 grados	leve	Motor:	adelante	PWM: 18 %
Inclinacion:	adelante	5 grados	leve	Motor:	adelante	PWM: 35 %
Inclinacion:	adelante	5 grados	leve	Motor:	adelante	PWM: 36 %
Inclinacion:	adelante	5 grados	leve	Motor:	adelante	PWM: 32 %
Inclinacion:	adelante	5 grados	leve	Motor:	detenido	PWM: 0 %
Inclinacion:	adelante	5 grados	leve	Motor:	adelante	PWM: 4 %
Inclinacion:	adelante	5 grados	leve	Motor:	adelante	PWM: 21 %
Inclinacion:	adelante	5 grados	leve	Motor:	adelante	PWM: 28 %
Inclinacion:	adelante	5 grados	leve	Motor:	adelante	PWM: 35 %
Inclinacion:	adelante	5 grados	leve	Motor:	detenido	PWM: 0 %
Inclinacion:	adelante	12 grados	leve	Motor:	adelante	PWM: 47 %
Inclinacion:	adelante	5 grados	leve	Motor:	adelante	PWM: 35 %
Inclinacion:	atras	-8 grados	leve	Motor:	reversa	PWM: 32 %
Inclinacion:	atras	-30 grados	fuerte	Motor:	reversa	PWM: 76 %
Inclinacion:	atras	-45 grados	fuerte	Motor:	reversa	PWM: 100 %
Inclinacion:	atras	-52 grados	fuerte	Motor:	reversa	PWM: 100 %
Inclinacion:	atras	-56 grados	fuerte	Motor:	reversa	PWM: 100 %
Inclinacion:	atras	-57 grados	fuerte	Motor:	reversa	PWM: 100 %
Inclinacion:	atras	-73 grados	fuerte	Motor:	reversa	PWM: 100 %
Inclinacion:	atras	-62 grados	fuerte	Motor:	reversa	PWM: 100 %
Inclinacion:	atras	-45 grados	fuerte	Motor:	reversa	PWM: 100 %
Inclinacion:	atras	-22 grados	moderada	Motor:	reversa	PWM: 70 %
Inclinacion:	adelante	30 grados	fuerte	Motor:	adelante	PWM: 18 %
Inclinacion:	adelante	22 grados	moderada	Motor:	adelante	PWM: 63 %
Inclinacion:	adelante	25 grados	moderada	Motor:	adelante	PWM: 68 %
ENTRA AL CENTRO: sensor dentro de +/-2 grados.						
Inclinacion:	centrado	2 grados	leve	Motor:	adelante	PWM: 37 %
SALE DEL CENTRO.						
Inclinacion:	atras	-50 grados	fuerte	Motor:	reversa	PWM: 49 %
Inclinacion:	atras	-56 grados	fuerte	Motor:	reversa	PWM: 100 %
Inclinacion:	adelante	6 grados	leve	Motor:	reversa	PWM: 58 %
ENTRA AL CENTRO: sensor dentro de +/-2 grados.						
Inclinacion:	centrado	0 grados	leve	Motor:	detenido	PWM: 0 %

Figura 3. Monitor Serie con inclinación, sentido del motor, PWM y avisos del centro.

6. Análisis de resultados

Los siguientes registros se tomaron de la Figura 3. El porcentaje representa el PWM aplicado al puente H; no es una medición de revoluciones por minuto del motor, porque el montaje no utiliza un encoder.

Cerca de 5° aparecen registros con el motor detenido y otros con giro. La decisión usa el ángulo con decimales, pero el Monitor Serie muestra grados redondeados. Por ejemplo, 4.7° se puede mostrar como 5° y seguir dentro de la zona muerta. El mínimo de 90 corresponde al **objetivo** fuera de esa zona; durante una rampa el PWM aplicado puede ser menor.

Tabla III. Ejemplos observados en la captura de funcionamiento.

Ángulo mostrado	Estado del motor	PWM	Interpretación
+12°	Adelante	47 %	Orden proporcional fuera de la zona muerta.
-8°	Reversa	32 %	Transición hacia un objetivo negativo.
-30°	Reversa	76 %	Mayor inclinación y mayor PWM aplicado.
-45° y -73°	Reversa	100 %	PWM limitado al máximo desde 45° de magnitud.
+30°	Adelante	18 %	PWM transitorio mientras aumenta la rampa.
+6°	Reversa	58 %	Desaceleración de una orden previa antes de invertir.
0°	Detenido	0 %	Motor sin orden de potencia al final de la captura.

Los valores transitorios explican por qué una inclinación positiva puede coincidir por un momento con el motor en reversa: el programa reduce primero el PWM anterior y cambia de sentido después de llegar a cero. Del mismo modo, al entrar al centro la rampa termina de reducir la orden antes de mostrar el motor detenido.

La calibración reduce el sesgo del giroscopio y el filtro combina la respuesta rápida del giro con la referencia de gravedad. Aun así, las sacudidas y aceleraciones intensas pueden alterar temporalmente la estimación. La rampa suaviza la orden eléctrica; llegar a PWM cero no implica que la rueda haya perdido instantáneamente toda su inercia [4, 5].

7. Preguntas de análisis

1. ¿Por qué no basta con usar solo el acelerómetro para medir la inclinación? Explica qué observaste al sacudir el sensor.

El acelerómetro registra la gravedad y también las aceleraciones del movimiento. Al sacudir el sensor, el ángulo calculado con esas lecturas puede cambiar bruscamente aunque la orientación apenas varíe. Combinarlo con el giroscopio ayuda a reducir esas variaciones [5].

2. ¿Qué ocurriría si se usara solo el giroscopio durante varios minutos?

Los pequeños errores se acumularían al integrar la velocidad de giro y el ángulo se desviaría, incluso con el sensor quieto. Ese efecto se llama deriva; el acelerómetro aporta una referencia para corregirlo.

3. ¿Qué función cumple la zona muerta de $\pm 5^\circ$? ¿Qué pasaría si fuera de 0° ?

Evita que pequeñas inclinaciones, ruido o vibraciones activen el motor. Con una zona de 0° , las variaciones alrededor del nivel provocarían arranques y cambios de sentido innecesarios.

4. ¿Por qué la velocidad mínima del motor es 90 y no 0?

El valor 90 es un PWM objetivo, aproximadamente 35 % de 255, que ayuda a vencer la fricción de los engranes. Con PWM 0 no se entrega potencia; con valores muy bajos el motor puede no arrancar. El mínimo útil también depende de la carga y la alimentación.

5. Explica dos problemas que evita la rampa de aceleración.

Reduce los cambios de corriente que pueden provocar caídas de alimentación y reinicios del Arduino. También disminuye los golpes mecánicos sobre los engranes, especialmente al pasar de adelante a reversa.

6. ¿Por qué el paro de seguridad se aplica de inmediato y no con rampa?

Porque una lectura fallida deja al sistema sin información confiable para controlar el motor. Usar una rampa mantendría la orden de movimiento durante más tiempo. Al detectar la falla se deshabilita el puente de inmediato, aunque la rueda todavía pueda moverse por inercia.

7. ¿Por qué el programa no usa delay()? ¿Qué pasaría con la respuesta del motor si se usara?

El ciclo principal usa millis() para atender sensor, motor, matriz y Monitor Serie sin bloquearse. Un delay() prolongado retrasaría la lectura y la respuesta del motor ante cambios o fallas. Sí hay esperas durante el arranque y la calibración, con el motor apagado.

8. Menciona tres aplicaciones reales de este sistema (por ejemplo: robots autobalanceados, plataformas niveladoras, controles por inclinación).

Puede servir como base para robots autobalanceados, plataformas que corrigen su nivel y controles que cambian la dirección o velocidad al inclinarse. Cada aplicación necesita adaptar los actuadores y la estrategia de control a su funcionamiento.

8. Conclusiones individuales

Ximena

Comprendí que el acelerómetro y el giroscopio se complementan para medir la inclinación con mayor estabilidad. Calibrar el sensor en reposo y usar el filtro ayudan a reducir errores en las lecturas. También entendí que una sacudida puede alterar el ángulo calculado, por lo que combinar ambos sensores mejora la relación entre la posición del MPU-6050 y el movimiento del motor.

Carlos Tadeo

Aprendí a controlar el sentido de giro y el PWM de un motorreductor mediante el puente H L298N. La zona muerta evita movimientos innecesarios cerca del nivel y la rampa suaviza los cambios de velocidad y dirección. Además, comprendí que el paro de seguridad es necesario para deshabilitar el motor cuando el sensor deja de proporcionar datos confiables.

Dulce Princesa

Esta práctica me ayudó a organizar varias tareas con millis() y a interpretar los avisos del Monitor Serie y la matriz LED. Aprendí que estos indicadores permiten revisar la inclinación, el estado del motor y las fallas de comunicación. También comprendí que el porcentaje de PWM representa la orden enviada al puente H y no una medición exacta de la velocidad.

Referencias

[1] Documento de la práctica proporcionado por el profesor: “3.2.3 Unidad de Medición Inercial”. Inclínómetro con control de motorreductor.

[2] InvenSense / TDK. *MPU-6000 / MPU-6050 Register Map and Descriptions*. Disponible en: invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Register-Map1.pdf

[3] Arduino. *Arduino UNO R4 WiFi User Manual*. Disponible en: docs.arduino.cc/tutorials/uno-r4-wifi/cheat-sheet/

[4] STMicroelectronics. *L298 Dual full-bridge driver*. Disponible en: st.com/resource/en/datasheet/l298.pdf

[5] Analog Devices. *AN-1057: Using an Accelerometer for Inclination Sensing*. Disponible en: analog.com/en/resources/app-notes/an-1057.html